
PROGRAMMING FUNDAMENTALS

1. What is programming?

- A computer ultimately understands instructions in the form of **machine language (0s and 1s)**. Humans, however, cannot conveniently write large programs using 0s and 1s.
- A program is a set of instructions given to a computer to perform a task.

There are two types of programming languages :

1. High – Level Programming Language
2. Low – Level Programming Language

LOW-LEVEL LANGUAGE

A **low-level language** is a language that is close to the computer's hardware and machine instructions.

There are two major types:

1. **Machine Language** : Machine language consists of **binary instructions**, usually represented using 0's and 1's.
2. **Assembly Language** : Assembly language uses **mnemonics** instead of writing binary instructions directly. (Ex: MOV AX, 10)

HIGH-LEVEL LANGUAGE

A high-level programming language is designed to be easier for humans to read, write, and understand.

Examples:

- Python
- C

Programming languages can also be classified based on how they are executed:

1. **Compiled Languages** : A compiler is a program that converts the entire program written in a programming language into machine code before the program is executed.
2. **Interpreted Languages** : An interpreter is a program that reads, translates, and executes a program during its execution.

PYTHON

What is Python?

Python is a high-level, general-purpose programming language used to develop different types of software and applications.

Python is widely used in:

- Web development
- Software development
- Data analysis
- Artificial Intelligence (AI)
- Machine Learning (ML)
- Automation ...

Features of Python:

- **Simple and Easy to Learn** – Python has a simple and beginner-friendly syntax.
- **High-Level Language** – Python hides complex hardware details from the programmer.
- **Interpreted** – Python code is executed through a Python interpreter/runtime.
- **Dynamically Typed** – You don't need to specify the variable's data type explicitly.
- **Object-Oriented** – Python supports programming using classes and objects.
- **General-Purpose** – Python can be used for many types of applications.
- **Cross-Platform** – Python programs can run on different operating systems.
- **Open Source** – Python is free to use and its source code is publicly available.
- **Large Standard Library** – Python provides many ready-to-use modules and functions.
- **Huge Package Ecosystem** – Thousands of third-party libraries and frameworks are available.
- **Readable Syntax** – Python code is easy for humans to read and understand.
- **Automatic Memory Management** – Python automatically manages memory for objects.

Disadvantages of python:

- **Slower execution speed**
- **High memory consumption**
- **Not ideal for mobile app development**
- **Higher runtime overhead**
- **Dynamic typing can cause runtime errors**

Variables and Data Types

- A **variable** is a name used to store or refer to a value in a program.
- In Python, a variable is created by assigning a value to a name using the = operator.
- **Syntax:** variable_name = value
- Example :
 - name = "Anu"
 - age = 20
 - is_student = True
- Python automatically determines the data type based on the assigned value.

Rules for Naming Variables

- Must start with a letter (a-z, A-Z) or underscore (_).
- Cannot start with a number and cannot contain spaces.
- Can contain letters, numbers, and underscores.
- Cannot use special characters such as @, #, \$, %, etc.
- Cannot use Python keywords as variable names.
- Variable names are case-sensitive.
- Use meaningful names whenever possible.

Data Type

- A data type defines the type of value that a variable can store or refer to.

Category	Data Type	Example	Description
Numeric	int	25	Whole numbers
Numeric	float	25.5	Decimal numbers
Numeric	complex	3 + 4j	Complex numbers
Sequence	str	"Hello"	Text or characters
Sequence	list	[1, 2, 3]	Ordered, changeable collection
Sequence	tuple	(1, 2, 3)	Ordered, unchangeable collection
Set	set	{1, 2, 3}	Collection of unique values
Mapping	dict	{"name": "Rahul"}	Collection of key-value pairs
Boolean	bool	True	Represents True or False
None	NoneType	None	Represents no value

Type Conversion

- Type conversion is the process of converting a value from one data type to another data type.
- Example :

```
age = 25
age = float(age)
print(age)                #output : 25.0
```

- There are two types of Type Conversion :
 - a. **Implicit Type Conversion** : Python automatically converts one data type into another.

Example :

```
num1 = 10
num2 = 5.5

result = num1 + num2

print(result)              #15.5
print(type(result))        #<class 'float'>
```

- b. **Explicit Type Conversion** : The programmer manually converts one data type into another. [int() float() str() bool() list() tuple() set()]

Example :

```
num = "25"

result = int(num)

print(result)              # 25
print(type(result))        # <class 'int'>
```

type() Function : The type() function is used to find the data type of a value or variable.

Example :

```
age = 25

print(type(age))
```